HyperCard を真似た論文作成 directory 構築

Shigeto R. Nishitani

2026 年 10 月 4 日

目次

1 Introduction 1
1.1 Introduction 2
1.2 An example : jps_26f 3
1.3 Preparation notes 3

2 Method 3
2.1 chunking, dejavu, HyperCard 4
2.2 preparation notes 4

3 Results 4
3.1 ppts 4
3.2 Pragmatic programmar (v2) 5
3.3 Why ruby? 5
3.4 下書き (preparation notes) 6

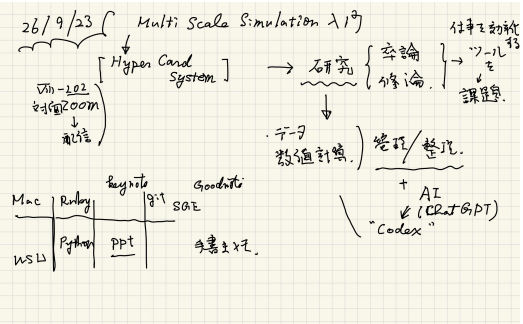
1 Introduction

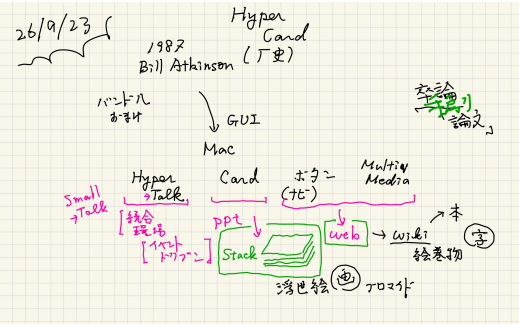
prev_button up_button next_button

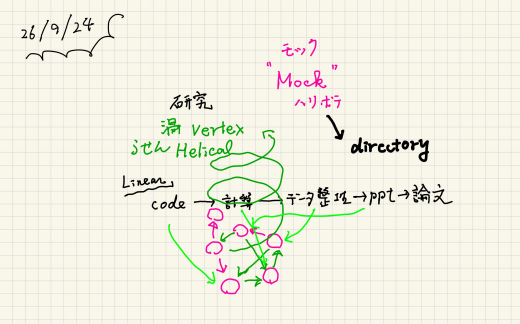
1.1 Introduction

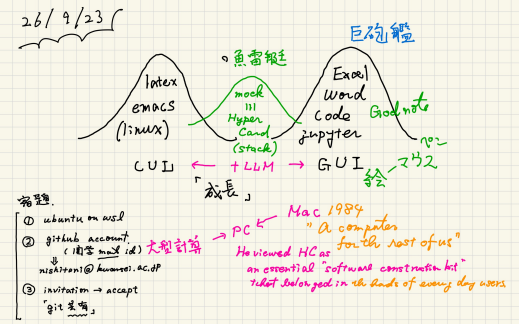
HyperCard っていう面白いソフトが初期の Mac には無料でついてました．これは，GUI 的なアプリの基本となってものです．でも，みんなが使えるアプリにするために，多くのいいところが削がれていきました．皆さんに提供しようとする HyperCard アプリはそれを"an essential software construction kit that belonged in the hands of everday users." とすることを目的としています．

HyperCard ではカードをまとめた"Stack"で Disk に入れて交換されていました．この授業では，院生，卒研生に利用してもらえる Mock-Up と作りながら"HyperCard"のコンセプトを理解してもらう予定です．

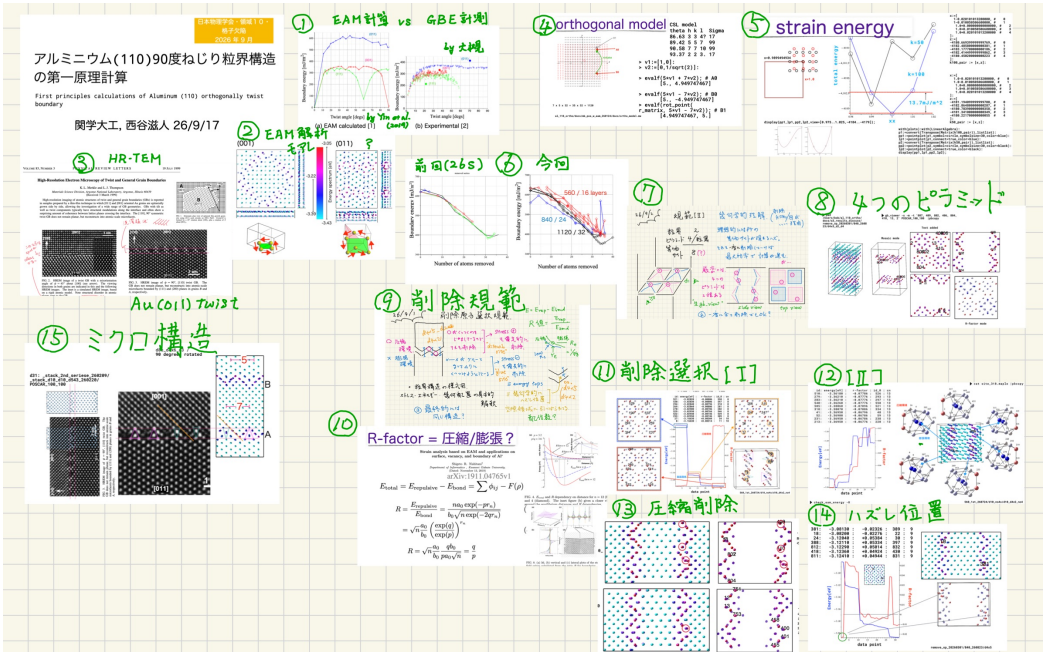








1.2 An example : jps_26f



これは，9 月に物理学会での発表の card です．これを作成するための Mock-Up をみていきましょう．

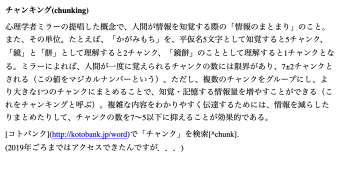
1.3 Preparation notes



2 Method

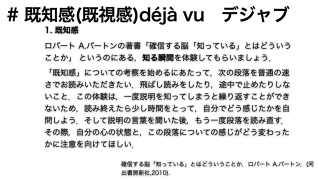
prev_button up_button next_button

2.1 chunking, dejavu, HyperCard


チャンキング(chunking)

心理学者ミラーの提唱した概念で、人間が情報を知覚する際の「情報のまとまり」のこと。また、その単位。たとえば、「かがみもち」を、平仮名5文字として知覚するとⅠチャンク、「鏡」と「餅」として理解するとⅡチャンク、「鏡餅」のこととして理解するとⅠチャンクとなる。ミラーによれば、人間が一度に覚えられるチャンクの数には限界があり、7±2チャンクとされる（この値をマジカルナンバーという）。ただし、複数のチャンクをグループにし、より大きな1つのチャンクにまとめることで、知覚・記憶する情報量を増やすことができる（これをチャンキングと呼ぶ）。複雑な内容をわかりやすく伝達するためには、情報を減らしたりまとめたりして、チャンクの数を7〜5以下に抑えることが効果的である。

[コトバンク](http://kotobank.jp/word)で「チャンク」を検索[*chunk].
(2019年ごろまではアクセスできたんですが．．．)

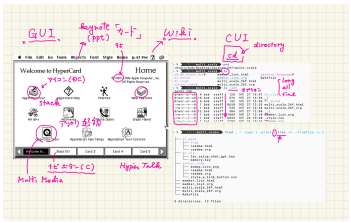

既知感(既視感)déjà vu　デジャブ

1. 既知感

ロバート A.バートンの著書「確信する脳「知っている」とはどういうことか」というのにある、知る瞬間を体験してもらいましょう。

「既知感」についての考察を始めるにあたって、次の段落を普通の速さでお読みいただきたい。飛ばし読みをしたり、途中で止めたりしないこと。この体験は、一度説明を知ってしまうと繰り返すことができないため、読み終えたら少し時間をとって、自分でどう感じたかを自問しよう。そして説明の言葉を聞いた後、もう一度段落を読み直す。その際、自分の心の状態と、この段落についての感じがどう変わったかに注意を向けてほしい。

確信する脳「知っている」とはどういうことか、ロバート A.バートン、(河出書房新社,2013).

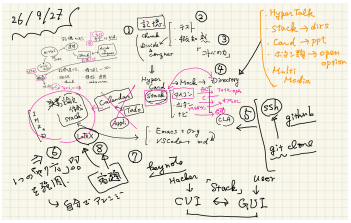


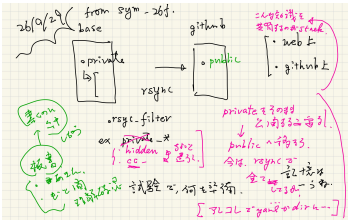
Mock-Up の基本になる考え方が chunking と deja vu です．chunking はまとまった知識を覚える人間の癖を心理学者が解き明かした知恵です．それをまとめる時にはコトバでもって概念としてまとめます．

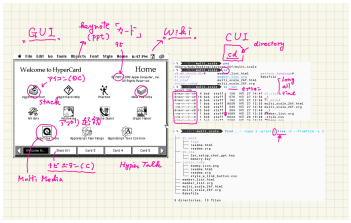
これは卒業研究の本質に深く根ざしています．それらの知識をまとめたのが次の文章です．

- `../d3_stack/hyper_card_philosophy_w_gemini.org`

2.2 preparation notes







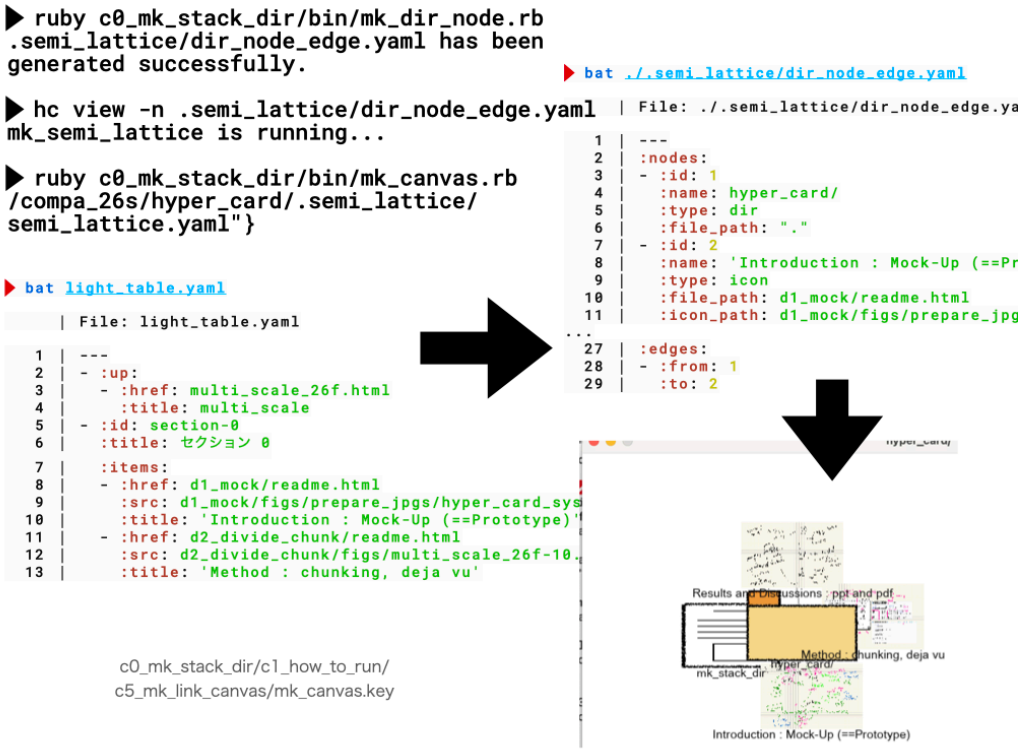
3 Results

prev_button up_button next_button

3.1 ppts

HyperCard の Mock が今見ていただいている http になります．

Icon とか Navi とか，Action が GUI と CUI でどのように実現されているかあるいは mock されているかを観察してください．



3.2 Pragmatic programmar (v2)

なぜ CUI が GUI よりプログラマに良いのかは，次のテキストを参照ください．

- basic tool
  - p3_basic_tools.pd
  - c16_plain_text.pdf
  - c17_shell.pdf
  - c18_editor.pdf
  - c19_version_github.pdf
  - c22_note_my_help.pdf

3.3 Why ruby?

さて，では，これらがどうやって実現されているか？そして everday users の手によってどのように構築されるかを示すのが次の目的になります．

以下のテキストにはその必要性が書かれています．

- c21_ruby.pdf
- c51_starter_kit_complete_auto.pdf

3.4 下書き (preparation notes)

